IoT 1

Les 4– Timer, SysTick, valversnelling via slinger,…

Timers

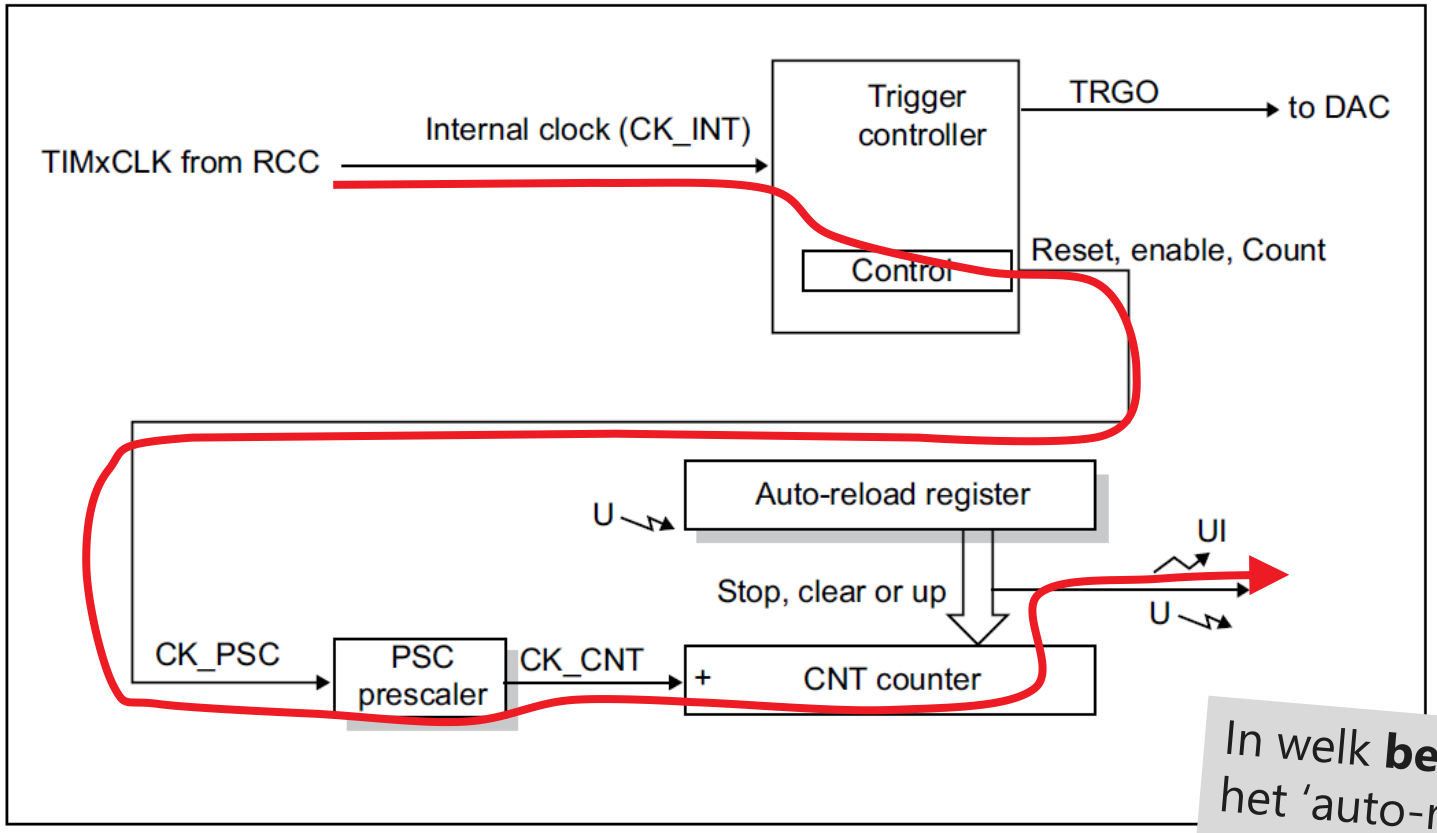
Timers

- **Wat**?
  - Een **digitale teller** die **hardwarematig** (met flipflops) ingebed zit in de microcontroller.
- **Voordeel**?
  - Kan **automatisch** tellen, **zonder tussenkomst van de CPU**!
  - Kan heel **nauwkeurig** tijd meten en **interrupts genereren**.
- Belangrijk
  - Hoe stabieler/nauwkeuriger de klok waarmee de timer gevoed wordt, hoe beter het resultaat…
  - Te vergelijken met een **zandloper**/keukenwekker.
  - **12** verschillende **timers** beschikbaar in de STM32F091RC!

Timers

Blokschema van Timer 6 en Timer 7.

Beide timers zijn gelijkaardig.



Notes:

Reg — Preload registers transferred to active registers on U event according to control bit

Event

Interrupt & DMA output

In welk **bereik** kunnen het 'auto-reload register' en de 'CNT counter' tellen?

Wat doet de '**prescaler**'?

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Timers

21.4.6 TIM6/TIM7 counter (TIMx_CNT)

Address offset: 0x24

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **CNT[15:0]**: Counter value

2^{16} = **65536** verschillende **mogelijkheden**, maar let op $(1111\ 1111\ 1111\ 1111)_2$ = $\mathbf{(65535)_{10}}$ is het **maximum** getal!

Bron: STM32F0 Reference Manual – RM0091 (st.com)

hogeschool vives

21.4.8 TIM6/TIM7 auto-reload register (TIMx_ARR)

Address offset: 0x2C

Reset value: 0xFFFF

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **ARR[15:0]**: Auto-reload value

ARR is the value to be loaded into the actual auto-reload register.

Refer to *Section 21.3.1: Time-base unit on page 548* for more details about ARR update and behavior.

The counter is blocked while the auto-reload value is null.

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Timers

21.4.7 TIM6/TIM7 prescaler (TIMx_PSC)

Let op: er is dus wel een **bovengrens** (16-bit getal).

Address offset: 0x28

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
1	0	1	1	1	0	1	1	0	1	1	1	1	1	1	1

Bits 15:0 **PSC[15:0]**: Prescaler value

The counter clock frequency CK_CNT is equal to f_{CK_PSC} / (PSC[15:0] + 1)

PSC contains the value to be loaded into the active prescaler register at each update event.

Hoe kan je de prescaler instellen om na **iedere milliseconde een klokpuls** te geven aan de teller? Dus CK_PSC = 48MHz => CK_CNT = 1kHz…

De inkomende klok moet 48000 keren gedeeld worden, daarvoor moet PSC op 47999 komen… Vul binair in, in bovenstaande tabel…

Bron: STM32F0 Reference Manual – RM0091 (st.com)

hogeschool vives

Timers

- Toepassingsvoorbeeld: **1/8ste secondeklok**

```
33    // Oneindige lus starten.
34    while (1)
35    {
36      // Lekker niets doen...
37    }
38  }
39
40  // Interrupt van Timer 6 opvangen.
41  void TIM6_IRQHandler(void)
42  {
43    if((TIM6->SR & TIM_SR_UIF) == TIM_SR_UIF)
44    {
45      // Interruptvlag resetten
46      TIM6->SR &= ~TIM_SR_UIF;
47
48      // LED's laten knipperen.
49      ToggleLed(1);
50      ToggleLed(8);
51    }
52  }
```

Doel: laat twee LED's knipperen met een tussentijd van **exact 125ms**...

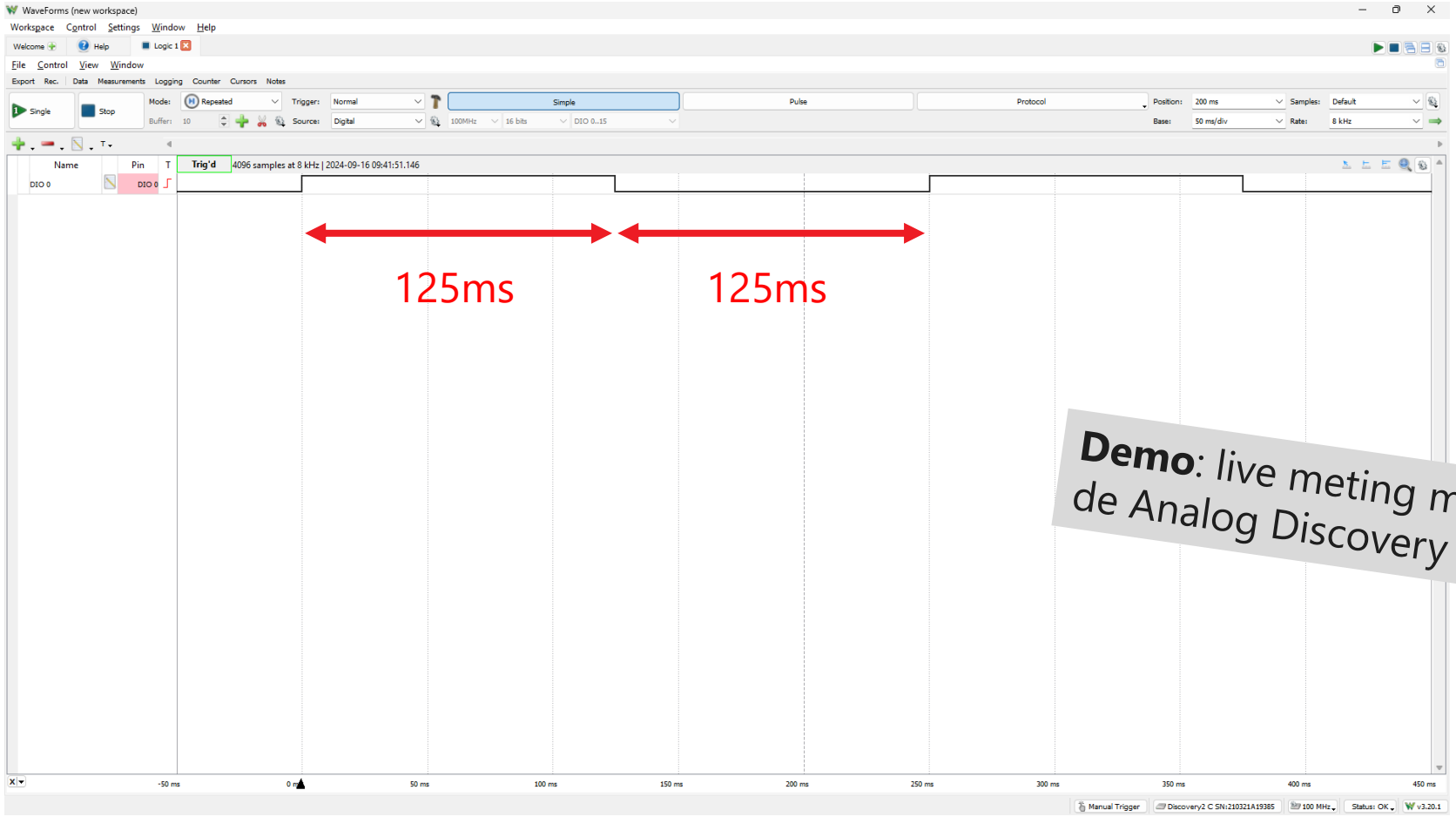
Timers

- Toepassingsvoorbeeld: **1/8ste secondeklok**

```c
void InitTimer6(void)
{
  RCC->APB1ENR |= RCC_APB1ENR_TIM6EN;    // De klok voorzien voor de timer.

  TIM6->PSC = 48000 - 1;                 // Als je een klok wil op 1000 Hz
                                         // = 48 000 000 / 48 000 = 1000 Hz
  TIM6->ARR = 125 - 1;                   // Als je iedere 125ms een interrupt wil...
                                         // 125 stappen van 1ms.

  TIM6->DIER |= TIM_DIER_UIE;            // Interrupt enable voor timer 6.
  TIM6->CR1 |= TIM_CR1_CEN;              // Counter enable.

  NVIC_SetPriority(TIM6_IRQn, 0);        // Kies een prioriteit.
  NVIC_EnableIRQ(TIM6_IRQn);             // Enable de interrupt.
}
```

Timers

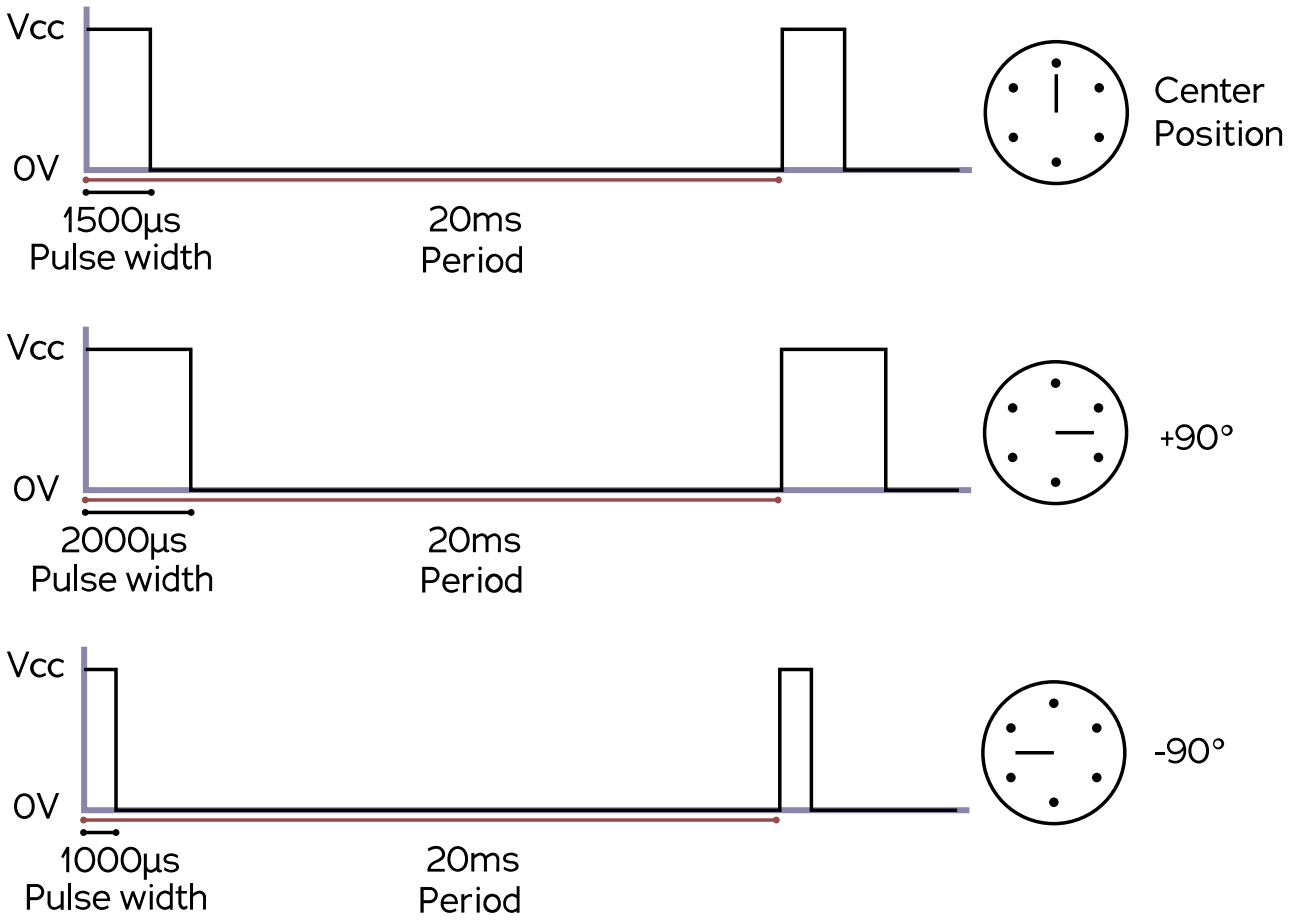
- Toepassingsvoorbeeld: **1/8ste secondeklok**



Demo: live meting met de Analog Discovery 2?!

Timers

- Toepassingsvoorbeeld: **Modelbouwservo PWM**



Bron: conrad.be en https://upload.wikimedia.org/wikipedia/commons/6/6c/Servomotor_Timing_Diagram.svg

Timers

Zoek op in de reference manual, waarom en hoe **PWM mode 1** ingesteld wordt...

- Toepassingsvoorbeeld: **Modelbouwservo PWM**

```
3  void InitPwm(void)
4  {
5    // Gebruik Timer 1 voor het genereren van een PWM-signaal. Mik op 50Hz frequentie en een resolutie van 10-bit.
6    // PWM aanmaken voor pin PA8 en PA9. PA8 is verbonden met LED6 van de Nucleo Extension shield
7    // en PA9 met een servomotor. Op die manier kan je ook visueel de PWM iet-of-wat controleren.
8    uint16_t pwm = 0;   // 1000 <= PWM <= 2000
9
10   // PA8/LED6 moet TIM1_CH1 worden via alternate function 2.
11   GPIOA->MODER = (GPIOA->MODER & ~GPIO_MODER_MODER8) | GPIO_MODER_MODER8_1;   // Alternate function op PA8.
12   GPIOA->AFR[1] |= 0x00000002;                                                // Alternate function 2.
13
14   // PA9/modelbouw ESC moet TIM1_CH2 worden via alternate function 2
15   GPIOA->MODER = (GPIOA->MODER & ~GPIO_MODER_MODER9) | GPIO_MODER_MODER9_1;   // Alternate function op PA9.
16   GPIOA->AFR[1] |= 0x00000020;                                                // Alternate function 2.
17
18   RCC->APB2ENR |= RCC_APB2ENR_TIM1EN;     // Clock voorzien voor de Timer1.
19   TIM1->PSC = 48 - 1;                     // Prescaler op 1/48 => 48000000/48 => 1 µs per puls.
20   TIM1->ARR = 20000 - 1;                  // Periode van 20 ms want: 1µs * 20000 = 20ms
21                                           // of  = 1/48000000 * 48 * 20000 = 20ms.
22   pwm = 1000;                             // Gewenste PWM-waarde (1000 <= pwm <= 2000).
23                                           // Want modelbouwservo verwacht minstens 1ms en maximum 2ms.
24   TIM1->CCR1 = pwm;                       // Aantijd voor OC1 instellen.
25   TIM1->CCR2 = pwm;                       // aantijd voor OC2 instellen.
26   TIM1->CCMR1 |= TIM_CCMR1_OC1M_2 | TIM_CCMR1_OC1M_1 | TIM_CCMR1_OC1PE;   // PWM mode 1 op OC1/PA8, enable preload register op OC1 (OC1PE = 1).
27   TIM1->CCMR1 |= TIM_CCMR1_OC2M_2 | TIM_CCMR1_OC2M_1 | TIM_CCMR1_OC2PE;   // PWM mode 1 op OC2/PA9, enable preload register op OC2 (OC2PE = 1).
28   TIM1->CCER |= TIM_CCER_CC1E | TIM_CCER_CC2E;                            // Enable OC1 (en OC2) output.
29   TIM1->BDTR |= TIM_BDTR_MOE;                                             // Enable output (MOE = 1).
30   TIM1->CR1 |= TIM_CR1_CEN;                                               // Enable counter (CEN = 1).
31   TIM1->EGR |= TIM_EGR_UG;                                                // Force update generation (UG = 1).
32 }
```


hogeschool vives

Timers

- Toepassingsvoorbeeld: **Modelbouwservo PWM**

```
// pwm = aantijd tussen 1000 en 2000 (µs).
void SetPwm(uint16_t pwm)
{
  TIM1->CCR1 = pwm;          // Aantijd voor OC1 instellen.
  TIM1->CCR2 = pwm;          // Aantijd voor OC2 instellen.
}
```

Eénmaal de *aantijd* ingesteld is in de PWM-module, hoeft de **CPU** zich daar NIETS meer van aan te trekken!

Ideaal, dat zorgt voor '**rust**' **of** tijd om **andere belangrijke zaken** te berekenen…

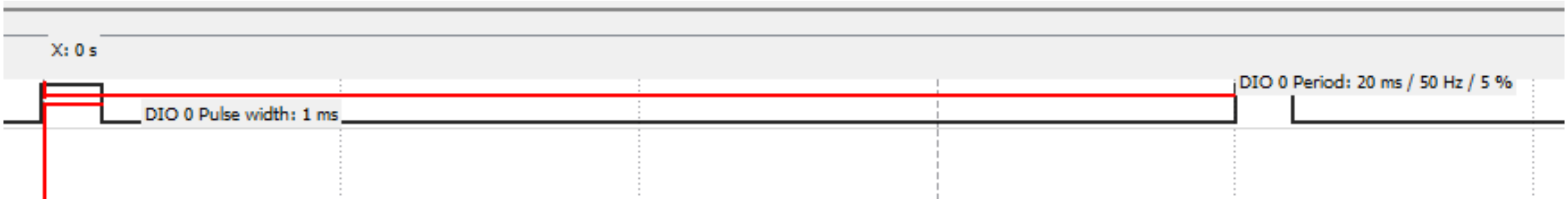
- Toepassingsvoorbeeld: **Modelbouwservo PWM**

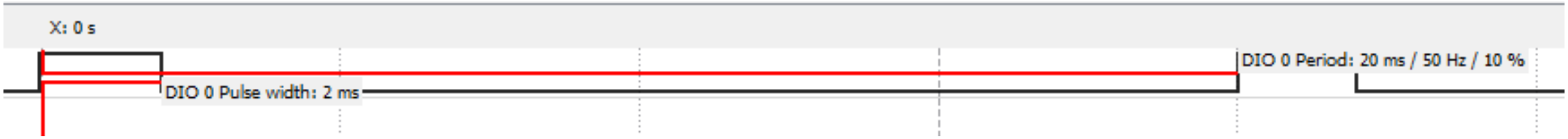
```
41    // Oneindige lus starten.
42    while (1)
43    {
44      // Een 12-bit AD-conversie doen en dan 2 bit opschuiven. Zodat je een getal tussen
45      // 0 en 1023 krijgt.
46      // De stand van de trimmer kan dan gebruikt worden om de stand van de servo
47      // in te stellen.
48      position = GetAdValue() >> 2;
49
50      // Waarde begrenzen.
51      if(position > 1000)
52        position = 1000;
53
54      // Servo instellen (met 1ms offset).
55      SetPwm(PWM_OFFSET + position);
56
57      // Even wachten (staat los van PWM-generatie).
58      WaitForMs(10);
59    }
60  }
```

In de **hoofdlus** kan je dan bijvoorbeeld de **AD-converter** aanspreken…

Timers

- Toepassingsvoorbeeld: **Modelbouwservo PWM**





Demo: nameten met de **USB-oscilloscoop** (via LED6 bijvoorbeeld)**?**

Timers

- Toepassingsvoorbeeld: **HC-SR04 ultra sonic sensor**

Deze ultrasone sensor, geeft aan hoelang een **geluidssignaal onderweg** is…

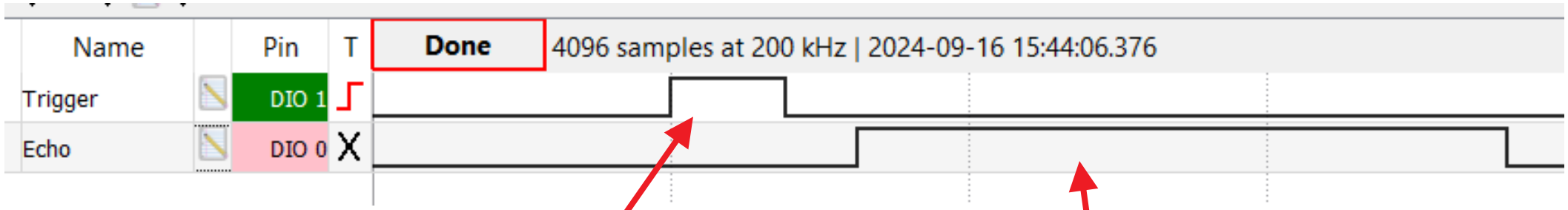
Als wij met onze microcontroller die **tijd** kunnen **opmeten**, kunnen we daaruit de **afstand** tot een object **berekenen**. Whoehooo!

Doe dat met Timer 14 die ingesteld staat als '**input capture mode**'.

Bron: https://res.cloudinary.com/rsc/image/upload/b_rgb:FFFFFF

Timers

- Toepassingsvoorbeeld: **HC-SR04 ultra sonic sensor**



Verzend een puls op te ‘trigger’ pin.

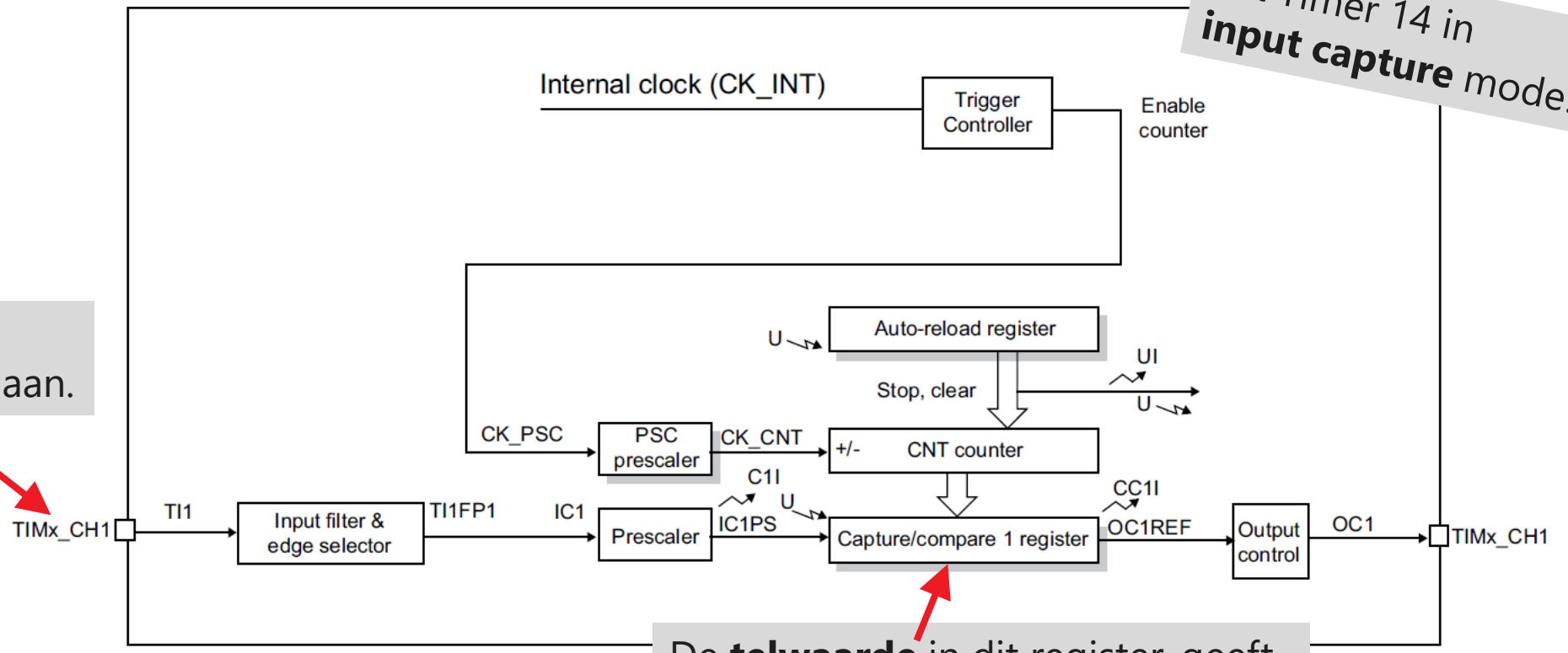
Ontvang een puls op de ‘echo’ pin, waarvan de lengte afhankelijk is van de afstand waarop zich een object bevindt.

Timers

- Toepassingsvoorbeeld: **HC-SR04 ultra sonic sensor**

Zet Timer 14 in **input capture** mode!

Verbind hier de ‘echo’ pin aan.



De **telwaarde** in dit register, geeft aan op welke afstand het object zich bevindt (in centimeters).

Bron: STM32F0 Reference Manual – RM0091 (st.com)

Timers

- Toepassingsvoorbeeld: **HC-SR04 ultra sonic sensor**

```
// Timer 14 instellen voor input capture.
void InitTimer14(void)
{
  // Klok voorzien voor Timer 14.
  RCC->APB1ENR |= RCC_APB1ENR_TIM14EN;

  // LET OP: onderstaande pin is niet 5V-tolerant! Gebruik een spanningsdeler!
  // PA7 = HC-SR04 echo = TIM14_CH1 instellen als alternate function.
  GPIOA->MODER = (GPIOA->MODER & ~GPIO_MODER_MODER7) | GPIO_MODER_MODER7_1;
  // TIM14_CH1 is alternate function AF4.
  GPIOA->AFR[0] |= 0x40000000;

  // CC1 kanaal als input zetten. IC1 wordt gelinkt aan TI1 (zie blokschema).
  TIM14->CCMR1 |= TIM_CCMR1_CC1S_0;
```

Timers

- Toepassingsvoorbeeld: **HC-SR04 ultra sonic sensor**

```
40    // Interrupts toelaten.
41    TIM14->DIER |= TIM_DIER_CC1IE;
42
43    // Prescaler instellen om rechtstreeks te meten in centimeters...
44    // 1cm (moet dubbel afgelegd worden) = 0.02m/340m/s = ~59 µs per centimeter.
45    // Gemeten met een klok van 48MHz is dit 59µs/20,8ns = ~2832 stappen
46    TIM14->PSC = 2832 - 1;
47
48    // Teller inschakelen.
49    TIM14->CR1 |= TIM_CR1_CEN;
50
51    // Koppeling van interrupt maken met de NVIC.
52    NVIC_SetPriority(TIM14_IRQn, 0);
53    NVIC_EnableIRQ(TIM14_IRQn);
54  }
```

Timers

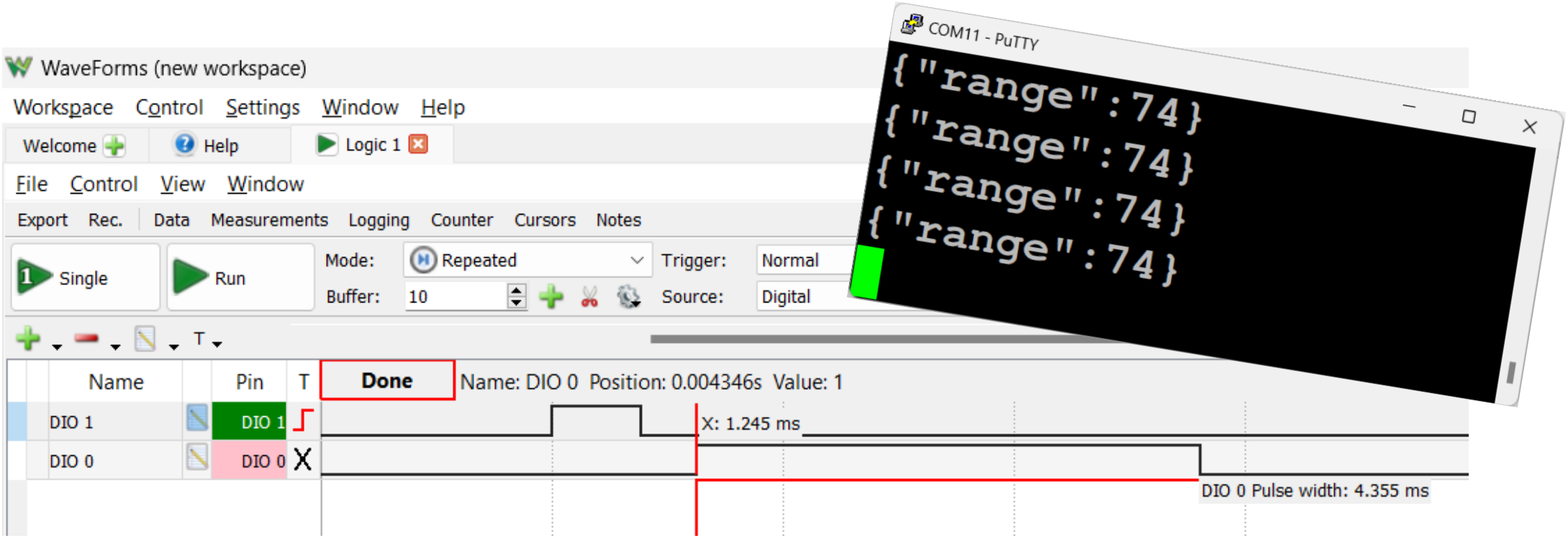
- Toepassingsvoorbeeld: **HC-SR04 ultra sonic sensor**

```c
// Interrupt van Timer 14 verwerken (input capture).
void TIM14_IRQHandler(void)
{
  // Lokale variabele.
  uint16_t temp = 0;

  // Interrutpt van Timer 14?
  if((TIM14->SR & TIM_SR_CC1IF) == TIM_SR_CC1IF)
  {
    // Capture compare interruptvlag resetten door het CCR1-register uit te lezen.
    temp = TIM14->CCR1;

    // Toestand van Echo pin bekijken (na spanningsdeler).
    if((GPIOA->IDR & GPIO_IDR_7) == GPIO_IDR_7)
    {
      // Stijgende flank gehad, teller resetten voor de meting.
      TIM14->CNT = 0;

      // Meting nu gevoelig maken aan dalende flank.
      TIM14->CCER &= ~TIM_CCER_CC1NP;
      TIM14->CCER |= TIM_CCER_CC1P;

      // Echotijd visualiseren (oplichten).
      SetUserLed(true);
    }
```

Demo van de code in µVision…

Timers

- Toepassingsvoorbeeld: **HC-SR04 ultra sonic sensor**



Effectieve meting hoogte **bureaublad**: 4355[µs]/59[µs/cm] = **74 cm…** Toppie.

SysTick

SysTick

- **Wat**?
  - Een **timer** die veel gebruikt wordt bij Real Time Operating Systems (RTOS) (zie module IoT 2).
  - Maar, kan **ook zonder RTOS** ingesteld en gebruikt worden.
- **Hoe**?
  - Via een **hardwarematige 24-bit afteller** op ritme van HCLK/8.
  - **Interrupt** bij bereiken van **nul**!
  - Hoge **prioriteit** (zie volgende slide)…

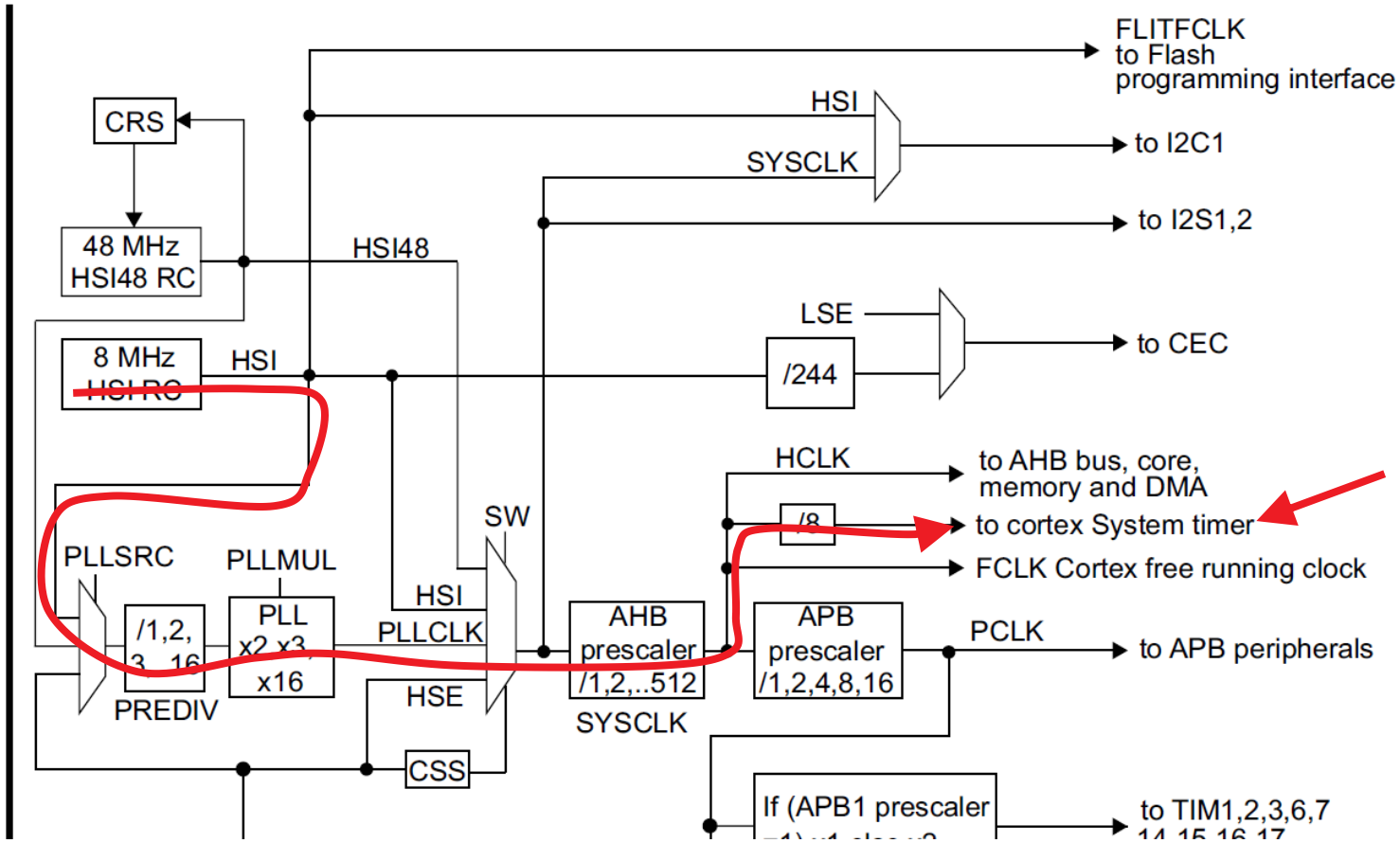
SysTick

Table 36. Vector table

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	fixed	Reset	Reset	0x0000 0004
-	-2	fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000 0008
-	-1	fixed	HardFault	All class of fault	0x0000 000C
-	3	settable	SVCall	System service call via SWI instruction	0x0000 002C
-	5	settable	PendSV	Pendable request for system service	0x0000 0038
-	6	settable	SysTick	System tick timer	0x0000 003C
0	7	settable	WWDG	Window watchdog interrupt	0x0000 0040
1	8	settable	PVD_VDDIO2	PVD and V_{DDIO2} supply comparator interrupt (combined EXTI lines 16 and 31)	0x0000 0044
2	9	settable	RTC	RTC interrupts (combined EXTI lines 17, 19 and 20)	0x0000 0048

SysTick

- **Blokschema**



Bron: STM32F0 Reference Manual – RM0091 (st.com)

SystTick

- **Praktisch** gebruik
    - Houd bij **hoelang** de microcontroller al is **opgestart**.
    - Doe dat in **milliseconden** via de SysTick timer en een variabele ***`ticks`***.
    - Gebruik die `ticks` ook in de ***`WaitForMs()`*** functie (indien gewenst).

SystTick

```c
// Handler die iedere 1ms opgeroepen wordt.
// Ingesteld met SystemCoreClockUpdate() en SysTick_Config().
void SysTick_Handler(void)
{
  ticks++;
}

// Wachtfunctie via de SysTick.
void WaitForMs(uint32_t timespan)
{
  uint32_t startTime = ticks;

  while(ticks < startTime + timespan);
}

// Klokken instellen. Deze functie niet wijzigen, tenzij je goed weet wat je doet.
void SystemClock_Config(void)
{
  //...

  SystemCoreClockUpdate();        // Nieuwe waarde van de core frequentie opslaan
                                  // in de SystemCoreClock variabele.
  SysTick_Config(48000);          // Interrupt genereren na iedere milliseconde.
                                  // Te verwerken via de SysTick_Handler() functie.
}
```

Als `ticks` een **32-bit unsigned** variabele is, **hoelang** kan die variabele dan betrouwbaar **gebruikt worden**?

Wat als je 'over' die tijd gaat?

hogeschool vives

Zelf de valversnelling meten via een slinger…

Valversnelling via slinger

- Wat is de **bedoeling**?
  - We willen **meten** hoe groot de **valversnelling** is…
  - Isaac Newton beschreef die ‘valversnelling’ voor het eerst, toen hij onder een appelboom zat en een **appel** zag vallen…

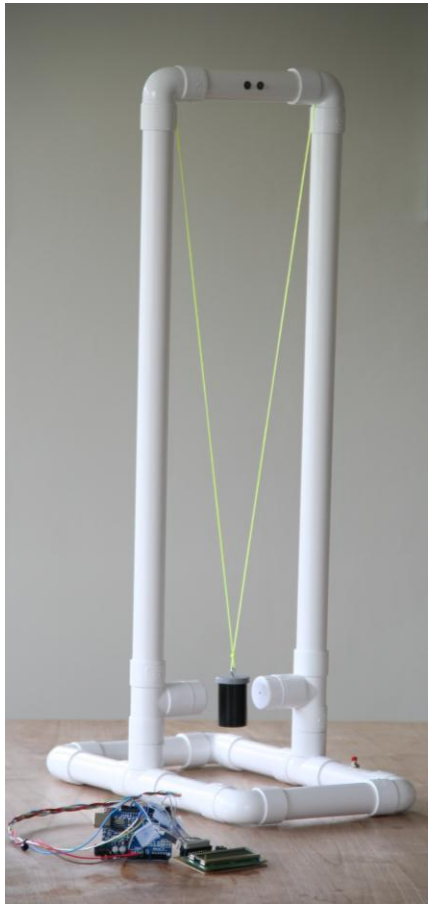
Bron: https://www.istockphoto.com/fr/vectoriel/newton

Valversnelling via slinger

- **Hoe**?

Gebruik een zelf gemaakte slinger en de **slingerformule** van Christiaan Huygens.

We zoeken dus: **g** **[m/s²]** .



$$T_{slinger} = 2.\pi.\sqrt{\frac{l}{g}} \quad \text{of} \quad g = 4.\pi^2.\frac{l}{T_{slinger}^2}$$

Als we de **periode** van één slingering kunnen **meten**, ligt de wereld aan ons voeten… ;-)

hogeschool vives

Valversnelling via slinger

- Wensen/**vereisten**?
    - **Draagbaar** systeem maken met **ongevaarlijke** slinger.
    - Een **sensor** gebruiken die weinig of **geen invloed** heeft op het slingeren.
    - **Snelle verwerking** van de gegevens.
    - **Feedback** via de virtuele seriële poort op de **computer**.
    - **Didactisch** interessante code gebruiken.

Valversnelling via slinger

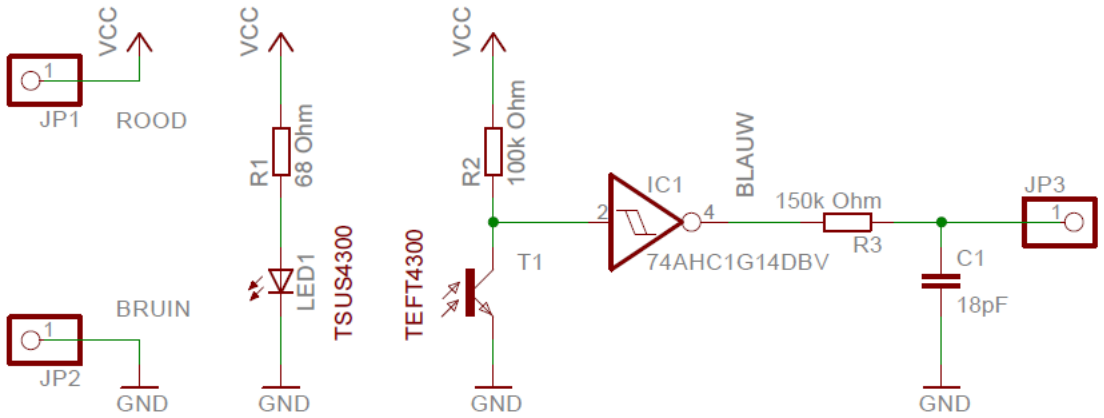
- **Hardware**?
  - PVC-buizen met daarin draadjes naar de LED's, drukknop en IR-sensor.
  - Slinger met touw en gewicht(je).
  - **Nucleo-F091** bord.
  - Laptop voor het uitlezen van de UART-data.

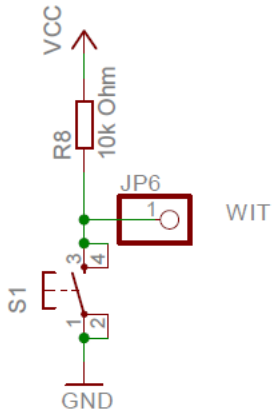
  - Meting zou **beter in het luchtledige** gebeuren, maar …

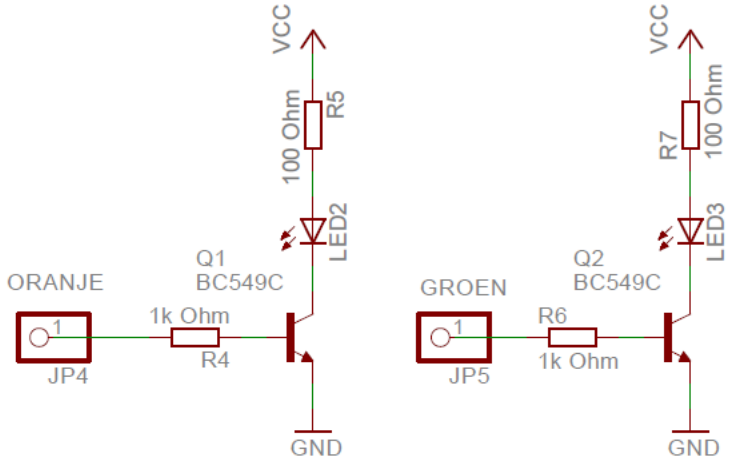
Valversnelling via slinger

- **Schema** interface?
  - Om de LED's, drukknop en IR-sensor te kunnen koppelen met het Nucleo-bord, is er een kleine 'interface' nodig. Een **'inter' – 'face'** bevindt zich tussen 'de mens' en 'de microcontroller'…
  - Schema op volgende slide.

Valversnelling via slinger







Bekijk de **werking** van:

- de actief lage **drukknop**.
- het aansturen van de **LED's** via een bipolaire **transistor**.
- een IR (infrarode) **lichtsluis**.
- een **Schmitt trigger**.
- een **RC-laagdoorlaatfilter**.

Valversnelling via slinger

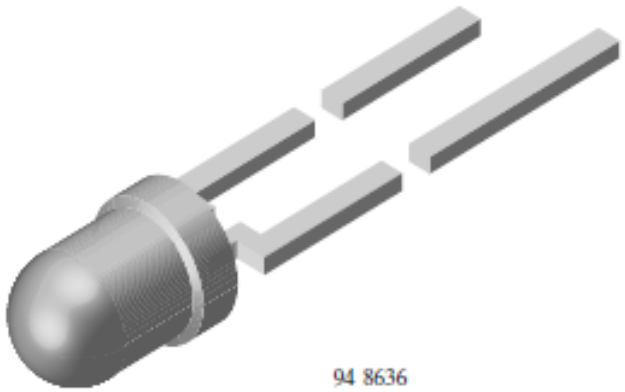
- **IR-LED** (TSUS4300)

Description

TSUS4300 is an infrared emitting diode in standard GaAs on GaAs technology, molded in a clear, blue tinted plastic package. Its lens provides a high radiant intensity without external optics.

Features

- High radiant power and radiant intensity
- Low forward voltage
- Suitable for DC and high pulse current operation
- Standard T-1(∅ 3 mm) package



Bron: datasheet TSUS4300.

Valversnelling via slinger

- **IR-LED** (TSUS4300)

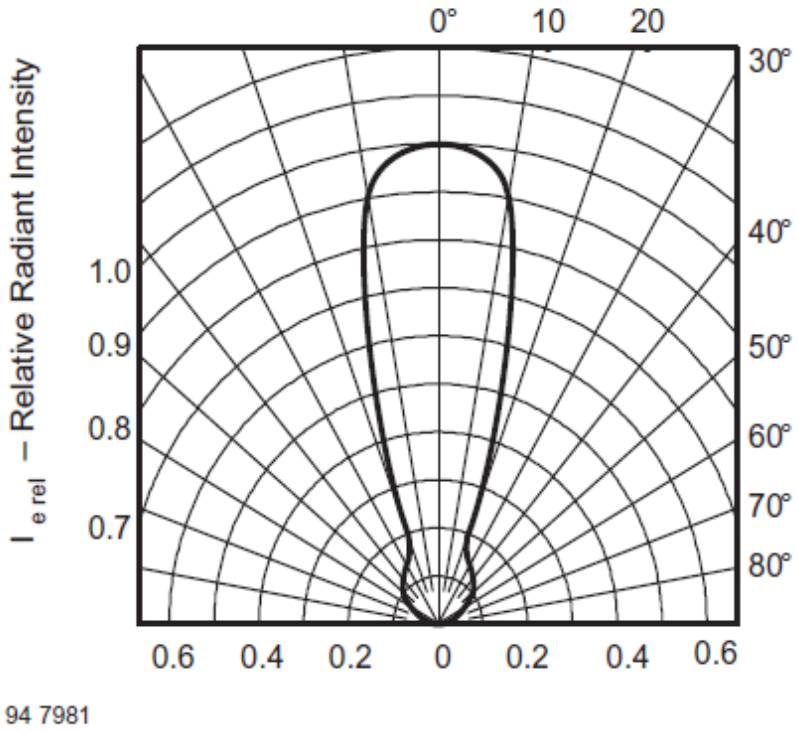
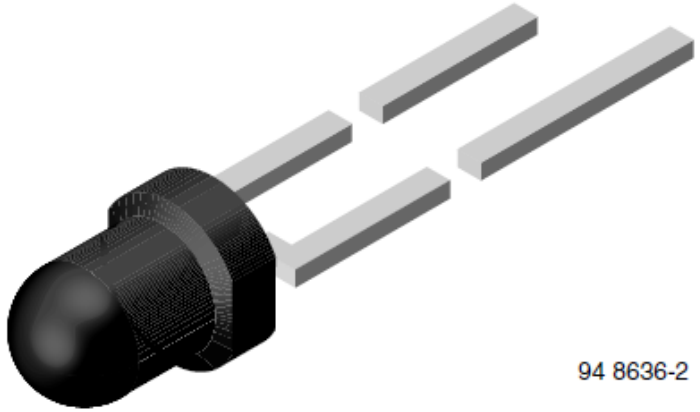


Figure 10. Relative Radiant Intensity vs. Angular Displacement

Bron: datasheet TSUS4300.

Valversnelling via slinger

- **Phototransistor** (TEFT4300)



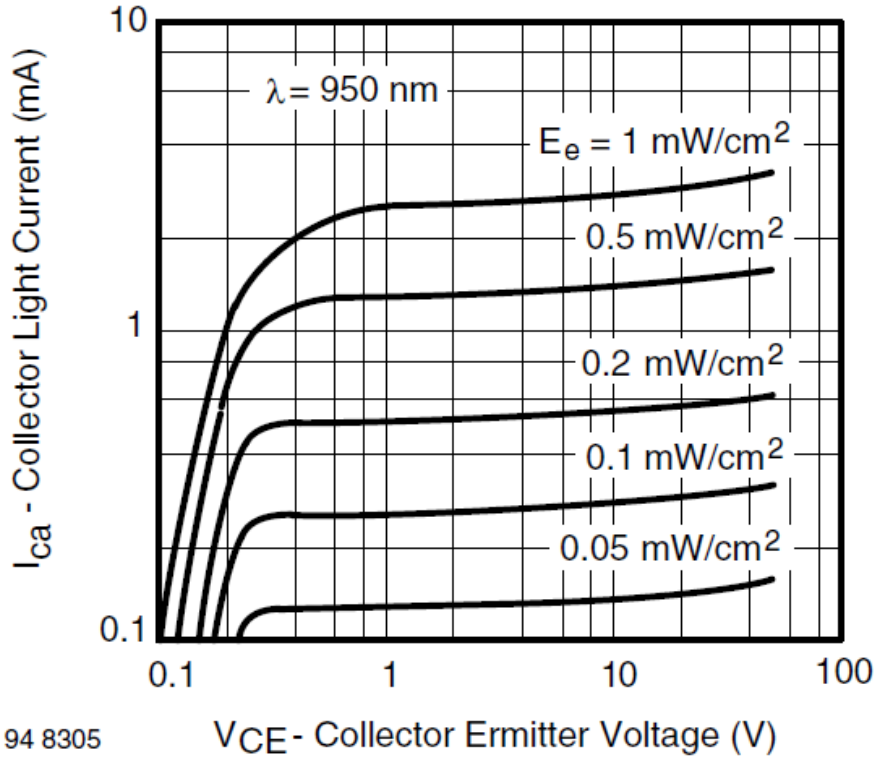
DESCRIPTION

TEFT4300 is a silicon NPN phototransistor with high radiant sensitivity in black, T-1 plastic package with daylight blocking filter. Filter bandwidth is matched with 900 nm to 950 nm IR emitters.

Bron: datasheet TEFT4300.

Valversnelling via slinger

- **Phototransistor** (TEFT4300)



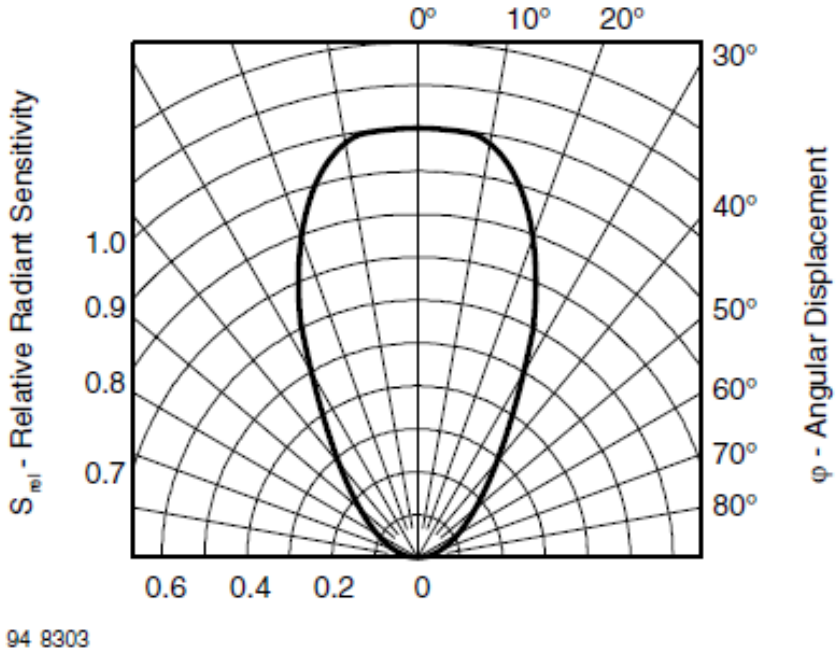


Fig. 9 - Relative Radiant Sensitivity vs. Angular Displacement

Bron: datasheet TEFT4300.

Valversnelling via slinger

- **Schmitt trigger** (741G14)

1 General description

The 74LVC1G14 provides the inverting buffer function with Schmitt-trigger input. It is capable of transforming slowly changing input signals into sharply defined, jitter-free output signals.

The input can be driven from either 3.3 V or 5 V devices. This feature allows the use of this device in a mixed 3.3 V and 5 V environment. Schmitt-trigger action at the input makes the circuit tolerant for slower input rise and fall time.

Bron: datasheet 741G14.

Valversnelling via slinger

- **Schmitt trigger** (741G14)

6 Functional diagram

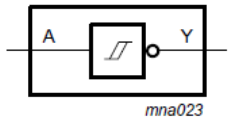


Figure 1. Logic symbol

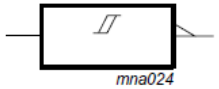


Figure 2. IEC logic symbol

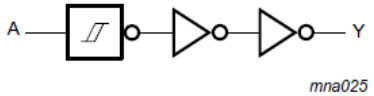


Figure 3. Logic diagram

7 Pinning information

7.1 Pinning

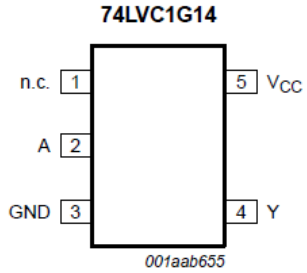


Figure 4. Pin configuration SOT353-1 and SOT753

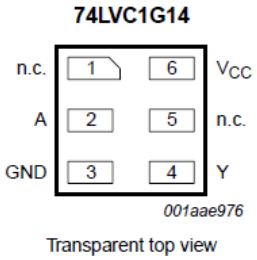


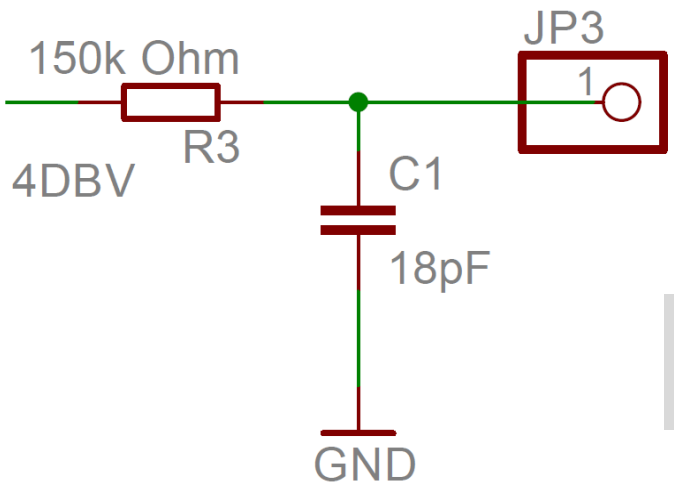
Figure 5. Pin configuration SOT886, SOT891, SOT1115 and SOT1202

Bron: datasheet 741G14.

Valversnelling via slinger

- **Laagdoorlaatfilter** (LDF)

$$f_c = \frac{1}{2.\pi.R.C} = \frac{1}{2.\pi.150.10^3.18.10^{-12}} \cong 59kHz$$



Het LDF kan mogelijke '**dender**' of jitter nog verder **wegwerken**…

Valversnelling via slinger

- **Firmware**

Stel de firmware **op**, met de globale vereisten in het achterhoofd…

```
58  // Oneindige lus starten.
59  while (1)
60  {
61    // Nieuwe cyclus starten als de oude voorbij is.
62    if(SwRedActive() && (currentAction == idle))
63    {
64      // Groene LED geeft aan dat de meting aan de gang is.
65      LedGreen(true);
66
67      // Variabelen resetten.
68      currentAction = startMeasuring;
69      startTicks = endTicks = differenceTicks = 0;
70      numberOfSwings = 0;
71
72      // Melding geven.
73      StringToUsart2("Start measuring.\r\n");
74    }
```

Valversnelling via slinger

- **Firmware**

Stel de firmware **op**, met de globale vereisten in het achterhoofd…

```
76      // Als de meting voorbij is, toon de info.
77      if(currentAction == doneMeasuring)
78      {
79        // Groene LED doven.
80        LedGreen(false);
81
82        // Melding geven.
83        StringToUsart2("Done measuring.\r\n");
84
85        // Periodetijd berekenen in milliseconden en melden.
86        differenceTicks = endTicks - startTicks;
87        sprintf(text, "Swing time: %d ms.\r\n", differenceTicks);
88        StringToUsart2(text);
89
90
91        // Aardversnelling meten met behulp van de formule van Christiaan Huygens en melden.
92        g = 4.0f * 3.1415f * 3.1415f * PENDULUM_LENGTH /
93            (((float)differenceTicks/1000.0f) * ((float)differenceTicks/1000.0f));
94        sprintf(text, "g = %.2f m/s2.\r\n\r\n", g);
95        StringToUsart2(text);
96
97        // Nieuwe meting mogelijk maken.
98        currentAction = idle;
99      }
```

Aangeven dat het type *float* gebruikt moet worden voor dit getal.

Valversnelling via slinger

- **Firmware**

Stel de firmware **op**, met de globale vereisten in het achterhoofd…

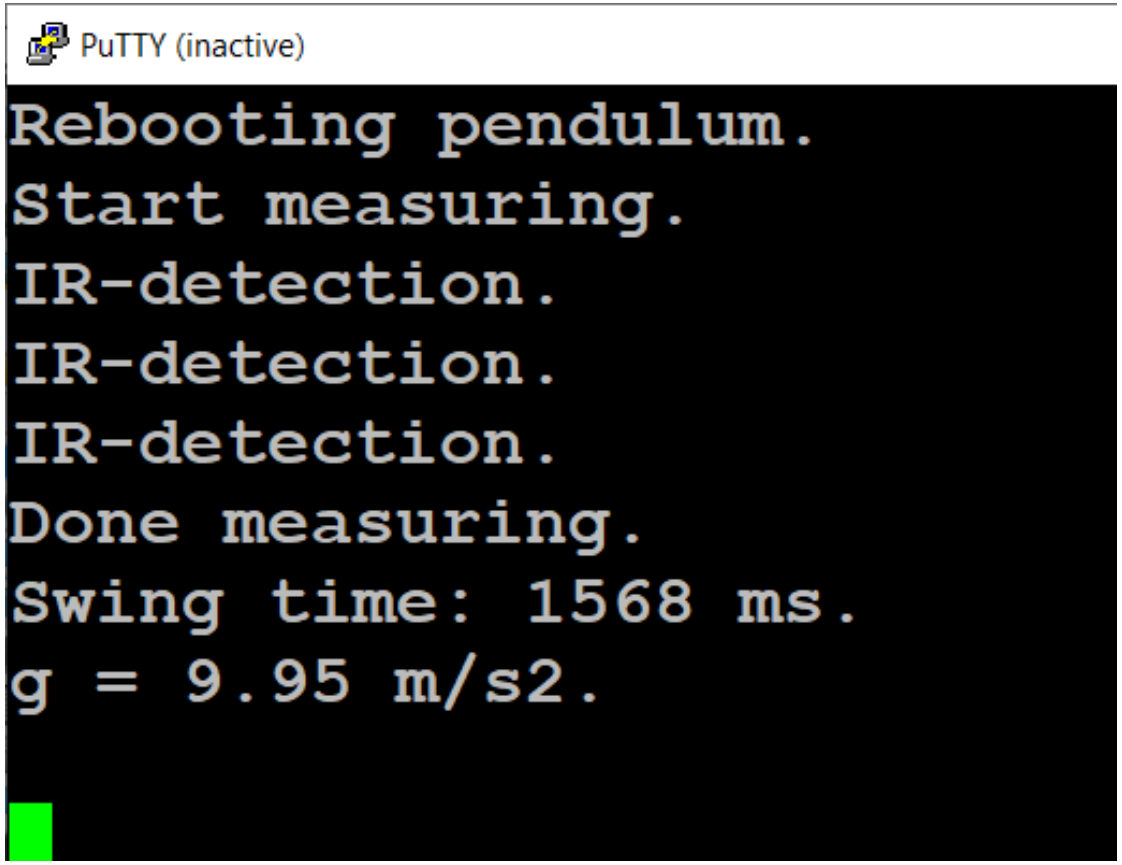
```
129     // UART-melding geven tijdens de meting.
130     if((currentAction == startMeasuring) || (currentAction == busyMeasuring))
131     {
132       StringToUsart2("IR-detection.\r\n");
133     }
134
135     // Meting uitvoeren en enumeratie updaten.
136     if(currentAction == startMeasuring)
137     {
138       // Meting start net, initialiseer de variabelen.
139       startTicks = ticks;
140       numberOfSwings++;
141       currentAction = busyMeasuring;
142     }
143     else
144     {
145       if(currentAction == busyMeasuring)
146       {
147         // Meting is bezig, verhoog het aantal slingeringen.
148         numberOfSwings++;
149
150         // Indien 3 slingeringen gezien, is er één volledige periode bemeten.
151         if(numberOfSwings == 3)
152         {
153           // Sluit de meting af.
154           currentAction = doneMeasuring;
155           endTicks = ticks;
156         }
157       }
158     }
```

Check het gebruik van enumeraties…

hogeschool vives

Valversnelling via slinger

- **UART-communicatie**



Your turn now!

- Is er nog **wat resterende** tijd?

  - Feedback voorbije theorieopdrachten?
  - Maak **Theorieopdracht 4 – output pin aansturen**.
  - Maak **Theorieopdracht 5 – input pin lezen**.

  - Ga **zelf aan de slag** om een ‘timer signaal’ op te meten met een Analog Discovery 2 **USB-oscilloscoop**.
  - Samen opdrachten/oefeningen maken?